

Java Backend Architecture

Interview Series

Chapter 17.10

Kubernetes Java API – Fabric8 Client

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 17.10 — Kubernetes Java API: Fabric8 Client

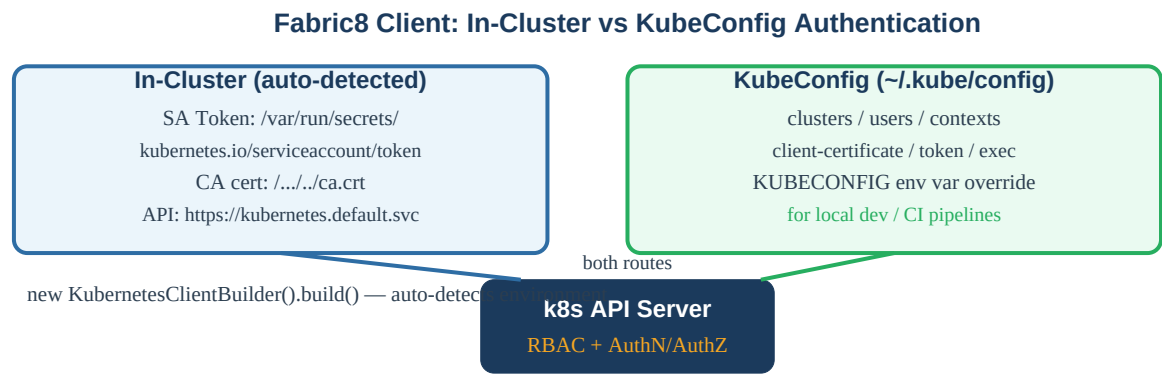
Overview

The Fabric8 Kubernetes Client is the most popular Java client for Kubernetes, providing a fluent DSL for interacting with the Kubernetes API. It supports in-cluster authentication, SharedIndexInformer for efficient caching, leader election, custom resources, and the Operator SDK.

Fabric8 vs Official Java Client

Feature	Fabric8 Client	Official Java Client (client-java)
Fluent DSL	Yes — fluent builder API	Verbose — raw ApiClient
Code generation	Limited (annotations)	openapi-generator based
Informers	SharedIndexInformer	SharedInformerFactory
Operator SDK	fabric8 operator-sdk	java-operator-sdk (multi-client)
Spring Boot	spring-boot-starter	manual integration
CRD support	client.resources()	CustomObjectsApi
Maturity	Very mature (10+ years)	Google-maintained, newer

Client Setup & Authentication



```
<!-- Maven dependency --> <dependency> <groupId>io.fabric8</groupId>
<artifactId>kubernetes-client</artifactId> <version>6.9.0</version> </dependency> //
Auto-detects: in-cluster SA token OR ~/.kube/config
KubernetesClient client = new
KubernetesClientBuilder().build(); // Explicit config
Config config = new ConfigBuilder()
.withMasterUrl("https://api.example.com")
.withAuthToken(token)
.withCaCertData(caCertBase64)
.build();
KubernetesClient client = new KubernetesClientBuilder()
.withConfig(config)
.build();
// Spring Boot auto-configuration
@SpringBootApplication
public class OperatorApp {
    @Autowired KubernetesClient client; // auto-configured
}
```

Common Client Operations

```
// List pods in namespace
PodList pods = client.pods().inNamespace("prod").withLabel("app", "api").list();
pods.getItems().forEach(pod ->
```

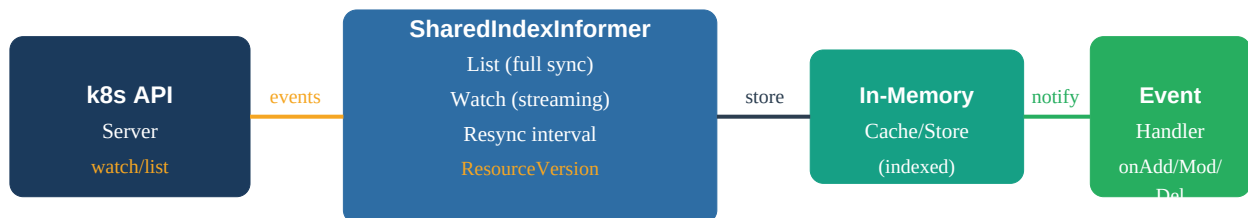
```
System.out.println(pod.getMetadata().getName()); // Get specific pod Pod pod = client.pods()
.inNamespace("prod").withName("api-pod-1").get(); // Create deployment Deployment deployment
= new DeploymentBuilder()
.withNewMetadata().withName("api").withNamespace("prod").endMetadata().withNewSpec()
.withReplicas(3).withNewSelector().addToMatchLabels("app", "api").endSelector()
.withNewTemplate().withNewMetadata().addToLabels("app", "api").endMetadata().withNewSpec()
.addNewContainer().withName("api").withImage("my-registry/api:v1.0")
.addNewPort().withContainerPort(8080).endPort().endContainer().endSpec().endTemplate()
.endSpec().build();
client.apps().deployments().inNamespace("prod").resource(deployment).create(); // Scale
deployment client.apps().deployments().inNamespace("prod").withName("api").scale(5); // 5
replicas // Delete pod client.pods().inNamespace("prod").withName("api-pod-1").delete(); //
Exec into pod ExecWatch exec = client.pods().inNamespace("prod").withName("api-pod-1")
.exec("sh", "-c", "ls /app");
```

Watch Resources

```
// Watch deployments (streaming, reconnect-aware) client.apps().deployments()
.inNamespace("prod").watch(new Watcher<Deployment>() { @Override public void
eventReceived(Action action, Deployment deployment) { String name =
deployment.getMetadata().getName(); switch (action) { case ADDED: System.out.println("Created:
" + name); break; case MODIFIED: System.out.println("Updated: " + name); break; case DELETED:
System.out.println("Deleted: " + name); break; case ERROR: System.out.println("Error: " +
name); break; } } @Override public void onClose(WatcherException e) { if (e != null) {
log.error("Watch closed with error", e); // Fabric8 auto-reconnects for HTTP 410
(ResourceExpired) } } }); // Note: Watcher reconnects on network errors; use Informer for
production
```

SharedIndexInformer — Efficient List-Watch

Fabric8 SharedIndexInformer: List-Watch Cache Architecture



`informer.getStore().list()` returns cached objects — no API call

Reduces API server load | Handles reconnect + re-list automatically

`InformerFactory.startAllRegisteredInformers()` — start all informers

```
// Create informer factory SharedInformerFactory factory = client.informers(); // Register pod
informer with 30s resync SharedIndexInformer<Pod> podInformer =
factory.sharedIndexInformerFor( Pod.class, PodList.class, 30 * 1000L // resync interval ms );
// Add event handler podInformer.addHandler(new ResourceEventHandler<Pod>() { @Override
public void onAdd(Pod pod) { log.info("Pod added: {}", pod.getMetadata().getName());
reconcile(pod); } @Override public void onUpdate(Pod oldPod, Pod newPod) { if
(!oldPod.getMetadata().getResourceVersion().
equals(newPod.getMetadata().getResourceVersion())) { reconcile(newPod); } } @Override public
void onDelete(Pod pod, boolean deletedFinalStateUnknown) { cleanup(pod); } }); // Start all
informers (non-blocking) factory.startAllRegisteredInformers(); // Wait for cache sync
podInformer.awaitCacheSync(); // blocks until initial list completes // Read from cache (no API
call) List<Pod> cachedPods = podInformer.getStore().list(); // Index: get pods by label (custom
indexer) podInformer.addIndexers(Map.of( "by-app", pod ->
```

```
List.of(pod.getMetadata().getLabels().getOrDefault("app", "")) )); List<Pod> apiPods =
podInformer.getIndexer().byIndex("by-app", "api");
```

Patch Strategies

- **JSON Merge Patch:** merges JSON — null removes field, partial update of top-level fields
- **Strategic Merge Patch:** k8s-aware merging (containers list merged by name, not replaced)
- **Server-Side Apply (SSA):** declarative, field-manager ownership, conflict detection
- **JSON Patch (RFC 6902):** explicit operations (add/replace/remove/move/copy/test)

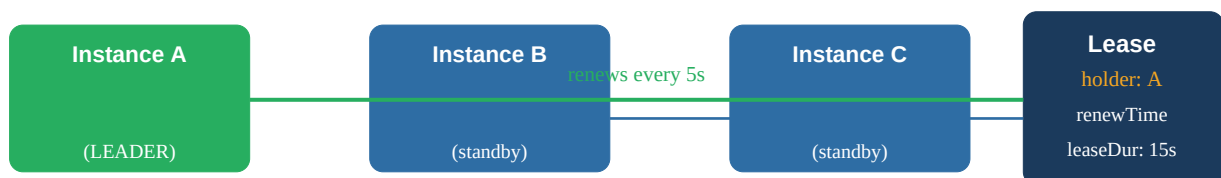
```
// JSON Merge Patch – update image tag
client.apps().deployments().inNamespace("prod")
.withName("api").patch(PatchContext.of(PatchType.JSON_MERGE_PATCH),
"{"spec":{"template":{"spec":{"containers":[{"name":"api","image":"api:v2.0"}]}}}"); //
Server-Side Apply – declarative ownership
Deployment desired = buildDesiredDeployment();
client.apps().deployments().inNamespace("prod").resource(desired).patch(new
PatchContext.Builder().withPatchType(PatchType.SERVER_SIDE_APPLY)
.withFieldManager("my-operator").withForce(true) // take ownership of conflicted fields
.build()); // JSON Patch – precise operation
client.pods().inNamespace("prod").withName("api-pod")
.patch(PatchContext.of(PatchType.JSON_PATCH),
"[{"op":"replace","path":"/spec/containers/0/image","value":"api:v2.0"}]");
```

Custom Resources

```
// Define Java model (with @Group, @Version, @Kind)
@Group("example.com") @Version("v1")
@Kind("MyApp") public class MyApp extends CustomResource<MyAppSpec, MyAppStatus> implements
Namespaced {} public class MyAppSpec { private String image; private int replicas; //
getters/setters } // Use typed client
MixedOperation<MyApp, KubernetesResourceList<MyApp>,
Resource<MyApp>> myAppClient = client.resources(MyApp.class); // List custom resources
MyApp myApp = myAppClient.inNamespace("prod").withName("my-app").get(); // Create/Update
myApp.getSpec().setReplicas(5);
myAppClient.inNamespace("prod").resource(myApp).createOrReplace(); // Watch custom resources
myAppClient.inNamespace("prod").watch(new Watcher<MyApp>() { @Override public void
eventReceived(Action action, MyApp resource) { reconcile(resource); } @Override public void
onClose(WatcherException e) {} });
```

Leader Election

Leader Election via Lease Renewal



If leader fails to renew within leaseDuration (15s): lease expires
 First standby to acquire lease (optimistic lock) becomes new LEADER
 k8s Lease API: coordination.k8s.io/v1 Lease in same namespace as controller

```
// Leader election for HA operator
LeaderElectionConfig leaderConfig = new
LeaderElectionConfigBuilder().withName("my-operator-leader").withNamespace("kube-system")
.withLeaseDuration(Duration.ofSeconds(15)).withRenewDeadline(Duration.ofSeconds(10))
.withRetryPeriod(Duration.ofSeconds(2)).withLeaderCallbacks(new LeaderCallbacks() { //
onStartLeading log.info("Became leader – starting reconciliation loop"); startController(); },
() -> { // onStopLeading log.warn("Lost leadership – stopping"); stopController(); }, identity
```

```
-> log.info("Current leader: {}", identity) )) .build()); LeaderElector elector = new
LeaderElector(client, leaderConfig); // Run in background thread
CompletableFuture.runAsync(elector::run);
```

Operator Pattern with Fabric8

- Operator = Informer (watch CRD) + Reconciler (desired vs actual state)
- fabric8 operator-sdk: @ControllerConfiguration, Reconciler<T> interface
- java-operator-sdk (JOSDK): community standard for k8s operators in Java
- Reconcile loop: get current state, compute diff, apply changes, update status
- Finalizers: prevent deletion until cleanup complete

```
// Spring Boot Operator with JOSDK @ControllerConfiguration(namespaces =
Constants.WATCH_ALL_NAMESPACES) public class MyAppReconciler implements Reconciler<MyApp> {
private final KubernetesClient client; @Override public UpdateControl<MyApp> reconcile(MyApp
myApp, Context<MyApp> context) { String name = myApp.getMetadata().getName(); String ns =
myApp.getMetadata().getNamespace(); int desired = myApp.getSpec().getReplicas(); // Check
current Deployment Deployment current = client.apps().deployments()
.inNamespace(ns).withName(name).get(); if (current == null) { // Create Deployment Deployment
deployment = buildDeployment(myApp);
client.apps().deployments().inNamespace(ns).create(deployment); } else { int actual =
current.getSpec().getReplicas(); if (actual != desired) { client.apps().deployments()
.inNamespace(ns).withName(name) .scale(desired); } } // Update status
myApp.getStatus().setAvailableReplicas(desired); return UpdateControl.patchStatus(myApp); } }
// Main Application @SpringBootApplication public class OperatorApplication { public static
void main(String[] args) { Operator operator = new Operator(); operator.register(new
MyAppReconciler(client)); operator.start(); } }
```

Interview Q&A; — Kubernetes Java API & Fabric8

Q1. What is the Fabric8 Kubernetes Client and why is it preferred over the official Java client?

Fabric8 is a community-maintained Java/Kotlin/Scala client with a fluent DSL for Kubernetes. Preferred because: fluent builder API (less boilerplate), rich informer/watch support, Operator SDK integration, Spring Boot starter, active community. Official client (client-java, Google-maintained): uses raw ApiClient, openapi-generated models, more verbose. java-operator-sdk works with both. For new projects: Fabric8 for Spring Boot operators; official client for generated API models.

Q2. How does Fabric8 auto-detect the Kubernetes environment?

KubernetesClientBuilder().build() uses AutoAdaptableKubernetesClient which checks in order: 1) In-cluster: if KUBERNETES_SERVICE_HOST env var set, reads /var/run/secrets/kubernetes.io/serviceaccount/token (projected SA token) and ca.crt. 2) KUBECONFIG env var path. 3) ~/.kube/config default location. 4) http://localhost:8080 fallback (kubectl proxy). RBAC still applies — client uses SA token which is subject to RBAC policies.

Q3. What is the difference between Watch and SharedIndexInformer?

Watch: simple streaming connection to API server. Must handle reconnection, ResourceExpired (410) errors, and re-list manually. No local cache — every read hits API server. Simple but fragile. SharedIndexInformer: built on watch but adds: local in-memory cache (Store), auto-reconnect and re-list on error, periodic resync, onAdd/onUpdate/onDelete callbacks. Production rule: always use Informer for controllers/operators, Watch only for one-off event streams.

Q4. How does the Informer handle reconnection after API server disconnect?

SharedIndexInformer maintains a ResourceVersion from the last received event. On disconnect, it reconnects and resumes watch from that ResourceVersion (no full re-list needed). If ResourceVersion is too old (HTTP 410 Gone), it does a full LIST to resync, resets ResourceVersion. This is transparent to the event handler — Informer manages reconnection automatically. The resync interval (configurable) triggers periodic full re-list for consistency.

Q5. What are the patch strategies available in Fabric8 and when do you use each?

JSON Merge Patch: simple JSON merge — null removes fields. Good for bulk field updates. Does not merge lists (replaces entire list). Strategic Merge Patch: k8s-native — merges containers by name (strategic key). Cannot be used on CRDs (CRDs are JSON merge only). Server-Side Apply: declarative — tracks field ownership per manager. Detects conflicts when two managers try to set the same field. Best for operators. JSON Patch (RFC 6902): precise operations. Use for single-field changes.

Q6. What is Server-Side Apply and why do operators prefer it?

Server-Side Apply (SSA) moves the apply logic to the API server. The client sends the full desired state; the server tracks which fields are 'owned' by which field-manager. Benefits for operators: 1) No conflict with kubectl (different field-managers). 2) Operator only needs to declare desired state, not diff. 3) Conflict detection: if operator and human both try to set replicas, server reports conflict. Use force: true when operator should take ownership of conflicted fields.

Q7. How do you handle Custom Resources in Fabric8?

Define Java model classes extending CustomResource<SpecType, StatusType> with @Group, @Version, @Kind annotations. Use client.resources(MyApp.class) to get a typed MixedOperation client. This supports: get, list, create, update, patch, delete, watch, informer. The CRD must exist in cluster (apply CRD YAML first or use Fabric8 CRD generator). Status sub-resource: UpdateControl.patchStatus(resource) updates only status (not spec).

Q8. What is the Operator pattern and how do you implement it with Fabric8?

Operator pattern: controller that watches a CRD (CustomResource) and reconciles cluster state to match the desired state in the CRD spec. Reconcile loop: onAdd/onUpdate event -> get current cluster state -> compare to desired spec -> apply changes (create/update/delete k8s resources) -> update CRD status. Fabric8 + JOSDK: implement `Reconciler<T>` interface, annotate with `@ControllerConfiguration`. The SDK handles: informer setup, event queuing, leader election, status updates.

Q9. How does leader election work and why is it needed for operators?

When deploying an operator with multiple replicas (for HA), only one instance should run the reconciliation loop at a time (single-writer principle). Leader election uses the Kubernetes Lease API (`coordination.k8s.io/v1`). The leader instance renews the Lease every `renewDeadline` period. If it fails to renew within `leaseDuration`, standby instances compete to acquire the Lease. First to write successfully (optimistic lock) becomes the new leader. Non-leaders wait in standby, ready to take over.

Q10. What are Finalizers in Kubernetes and how do you use them in operators?

Finalizers are strings on object metadata that prevent deletion until removed. When `kubectl delete` runs: object gets `deletionTimestamp` set (soft delete). Controller detects `deletionTimestamp`, performs cleanup (delete external resources, revoke permissions, archive data). Then removes finalizer: `resource.getMetadata().getFinalizers().remove("myapp.example.com/cleanup")`. `kubectl apply/patch` to update. Once finalizers list is empty, k8s hard-deletes the object. Without finalizer: deletion is immediate and cleanup cannot happen.

Q11. How do you handle rate limiting in the reconciliation loop?

The JOSDK and Fabric8 SDK use work queues with rate limiting to prevent reconciler overload. Multiple events for the same resource are deduplicated (only one reconciliation per resource in the queue at a time). Rate limiting: exponential backoff on error (`RetryPolicy`). Implement: return `UpdateControl.noUpdate()` when nothing changed (avoid status update storm). Compare `generation/resourceVersion` before acting. Add `workqueue` metrics for observability.

Q12. What is the informer cache consistency model?

The informer cache is eventually consistent with the API server. List-Watch guarantees: once an event is returned by List, subsequent Watch events are ordered and complete. Cache is consistent after the initial sync (`awaitCacheSync()`). Never use both cache (`informer.getStore()`) AND direct API calls for the same resource — they can be momentarily inconsistent. For operators: always read from cache; write directly to API server.

Q13. How do you test Kubernetes operators written with Fabric8?

1) `KubernetesMockServer` (okhttp mock): records/replays HTTP interactions in unit tests. 2) `EnvTest` (controller-runtime inspired): starts a real API server (etcd + apiserver binary) for integration tests. 3) Minikube/Kind-based integration tests: deploy operator, create CR, assert behavior. 4) JOSDK's `KubernetesClientMockServer`: mock client for operator unit tests. Test `reconcile()` directly with mock client — verify correct API calls made.

Q14. What is the SharedInformerFactory and how do you use it efficiently?

`SharedInformerFactory` manages multiple informers and shares a single watch connection per resource type. Calling `sharedIndexInformerFor(Pod.class)` multiple times returns the SAME informer instance. Start with `factory.startAllRegisteredInformers()` then wait for `factory.waitForCacheSync()`. Namespace-scoped: `factory = client.informers()` uses all namespaces. For namespace-specific: `client.inNamespace('prod').informers()`. Stop informers cleanly: `factory.stopAllRegisteredInformers()` on shutdown.

Q15. How do you handle status updates in custom resources?

Status sub-resource (enabled in CRD spec: `subresources.status`) separates spec and status updates. Updating spec requires standard PATCH/PUT on resource. Updating status requires PATCH/PUT on resource/status endpoint. In Fabric8: `UpdateControl.patchStatus(resource)` patches only status — prevents spec changes from triggering infinite reconcile loops. Printer columns in CRD: display status fields in `kubectl get` output.

Q16. What is the difference between Reconciler and EventHandler in operator patterns?

EventHandler (Informer): `onAdd/onUpdate/onDelete` callbacks — raw stream of events. No deduplication, no retry, developer manages all error handling. Reconciler (Operator SDK): level-driven, not edge-driven. Reconciles CURRENT STATE vs DESIRED. Even if 10 events arrive, only one reconcile runs (deduplicated). Reconciler is idempotent by design — safe to re-run. Always implement operators as reconcilers, not event handlers — more resilient.

Q17. How does Fabric8 handle connection pooling and TLS?

Fabric8 uses OkHttp under the hood. Connection pooling: configurable via `ConnectionPoolConfig`. TLS: uses JVM trust store or explicit `caCertFile/caCertData` config. Mutual TLS: `clientCert` + `clientKey` for authentication. HTTP/2: supported by OkHttp — multiplexes requests on single connection (important for watch streams). Custom TrustManager: `ConfigBuilder.withTrustCerts(true)` (insecure, dev only!). Timeout tuning: `connectTimeout`, `readTimeout`, `writeTimeout`, `websocketTimeout` for watches.

Q18. How do you implement retry logic in a Fabric8 operator?

JOSDK `RetryPolicy`: configure `maxAttempts` and backoff intervals per resource type. Custom: return `UpdateControl.rescheduleAfter(Duration.ofSeconds(30))` from `reconcile()`. Error handling: catch exceptions, log, return empty `Optional` to trigger requeue. Exponential backoff: start with 1s, double up to `maxDelay` (e.g., 5 minutes). Permanent failures: update `status.conditions` with Failed status; don't retry indefinitely — alert on repeated failures.

Q19. What is the role of ResourceVersion in Fabric8 watch/informer?

`ResourceVersion` is a server-assigned opaque string identifying the cluster's state at a point in time. Watch from `resourceVersion`: only receive events after that version (no replay). `ResourceVersion=0` in list request: may return stale cache (fast). `ResourceVersion=""` (empty): always reads from etcd (consistent but slower). Informer stores `lastSyncResourceVersion` and uses it to resume watch after disconnect. Operators: never parse `ResourceVersion` — treat as opaque (may change format).

Q20. How do you use the Fabric8 client in a Spring Boot application?

Add `fabric8 spring-boot-starter` dependency. Auto-configures `KubernetesClient` bean. Properties: `spring.cloud.kubernetes.client.namespace`, `master-url`, `trust-certs`. Inject: `@Autowired KubernetesClient client`. For operators: add `java-operator-sdk spring-boot-starter`. Beans annotated with `@ControllerConfiguration` are auto-registered. `fabric8` handles lifecycle in Spring context. Testing: `@SpringBootTest` with `@MockBean KubernetesClient` for unit tests.

Q21. What are the performance considerations for Fabric8 informers at scale?

Memory: informer cache holds all objects in memory. 10k pods x 5KB each = 50MB per informer. Limit by namespace or label selector: `sharedIndexInformerFor` with `ListOptions`. Watch connections: each informer type = one persistent HTTP/2 stream. Resync interval: too short = unnecessary CPU processing; 30-300s is typical. Work queue: use JOSDK's built-in queuing to prevent reconciler contention. Metrics: expose informer lag, queue depth, reconcile duration via `Micrometer/Prometheus`.

Q22. What is the difference between inNamespace() and inAnyNamespace()?

`inNamespace('prod')`: operations scoped to 'prod' namespace. `inAnyNamespace()`: operations across ALL namespaces (cluster-scoped). Requires `ClusterRole` (not just `Role`) with appropriate verbs. Informers: `client.informers()` creates factory watching all namespaces. `client.inNamespace('prod').informers()` watches only 'prod'. Best practice: use namespace-scoped informers when possible to reduce API server load and limit blast radius of operator bugs.

Q23. How do you handle the Operator lifecycle (start, stop, health)?

Start: `factory.startAllRegisteredInformers()`, `operator.start()` — non-blocking. Wait for cache sync before starting reconcilers: `informer.awaitCacheSync()`. Health probes: implement readiness probe that checks `informer.hasSynced()` — not ready until cache populated. Liveness: check leader election status. Graceful shutdown: `factory.stopAllRegisteredInformers()`, leader election stop, drain in-flight reconciliations (`await CompletableFuture`). Register ShutdownHook: `Runtime.getRuntime().addShutdownHook` for JVM SIGTERM handling.

Q24. What is the Kubernetes informer resync mechanism?

Resync is a periodic timer (`resyncPeriod`) that triggers `onUpdate` callbacks for ALL cached objects, even if nothing changed on the API server. Purpose: re-drive reconciliation for objects that may have drifted but generated no events. Example: external system modifies a resource outside k8s — no k8s event generated. Resync detects this on next cycle. Set to 0 to disable. Typical value: 30s-10min based on how critical drift detection is. In operator reconcile: check `Generation` to avoid redundant processing during resync.

Q25. How do you use OwnerReferences in operators?

`OwnerReferences` create a parent-child relationship between resources. When the owner (e.g., `MyApp CR`) is deleted, k8s garbage-collects all owned resources (`Deployment`, `Service`, `ConfigMap` created by the operator). Set with: `new OwnerReferenceBuilder().WithName(cr.getName()).withUid(cr.getMetadata().getUid()).withKind(cr.getKind()).withApiVersion(cr.getApiVersion()).withController(true).build()`. `blockOwnerDeletion: true` — prevents owner deletion until owned resources are deleted first.

Q26. What debugging tools are available for Fabric8 operators?

1) Enable `DEBUG` logging for `io.fabric8` logger — logs all API requests/responses. 2) `client.pods().inNamespace(ns).withName(name).getLog()` — read pod logs programmatically. 3) `kubectl get events --field-selector involvedObject.name=my-resource`. 4) `JOSDK` event sources: log reconcile queue state. 5) `Micrometer` metrics: `reconcile.success/failure/duration` histograms. 6) Trace HTTP with `OkHttp HttpLoggingInterceptor`. 7) `kubectl describe` — check `status.conditions` set by operator.